SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)

Name: Abhishek Pandey SAP ID: 60019220110
Roll No: B073 Branch: CSE(ICB)

EXPERIMENT 2

Aim: Conduct code reviews and static analysis to identify potential security flaws such as re-entrancy bugs, integer overflows, or unchecked user inputs in smart contract.

Theory:

- **Effectiveness of Detection:**
  - How effective are manual code reviews and static analysis tools in identifying re- entrancy bugs, integer overflows, and unchecked user inputs in smart contracts?
- **Tool and Review Challenges:**
  - What are the primary challenges and limitations faced when using static analysis tools and conducting manual code reviews for security vulnerabilities?
- **Accuracy and False Positives:**
  - How often do you encounter false positives or negatives with static analysis tools, and how do these compare with the findings from manual code reviews?
- **Coverage and Integration:**
  - How comprehensive is the coverage of static analysis tools, and how well do they integrate into your development workflow for smart contracts?
- **Comparative Efficiency:**
  - How does the time efficiency of manual code reviews compare to static analysis tools in identifying security flaws in smart contracts

Q 1: Implement Integer Overflow/Underflow Vulnerability:

Code:

1. Code a vulnerable smart contract ERC20.sol.

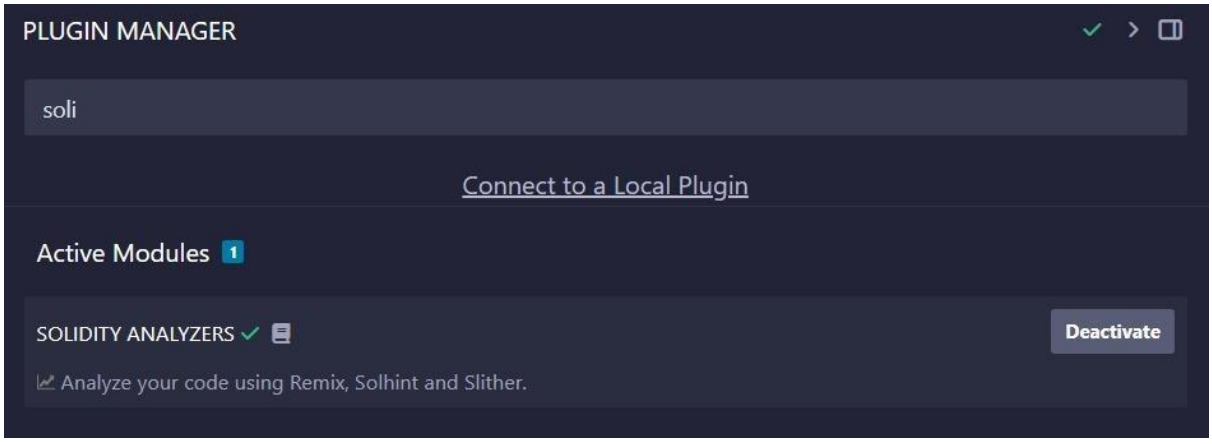
Name: Abhishek Pandey
Roll No: B073
SAP ID: 60019220110
Branch: CSE(ICB)

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

contract MyToken is ERC20 {
    constructor() ERC20("MyToken", "MT") {    infinite gas 879800 gas
        _mint(msg.sender, 1000000 * (10                    ()));
    }

    // Vulnerable withdraw function
    function withdraw(uint256 amount) p                    gas
        require(balanceOf(msg.sender) >= amount, "Insufficient balance");

        // Reentrancy vulnerability: External call before state change
        (bool success, ) = msg.sender.call{value: amount}("");
        require(success, "Transfer failed");

        // State change happens after the external call
        _burn(msg.sender, amount);
    }

    // Overflow vulnerability: Subtracting more than the balance
    function burnTokens(uint256 amount) public {    infinite gas
        // No check to ensure 'amount' is less than or equal to the balance
        _burn(msg.sender, amount);
    }

    // Unchecked user input: No validation of recipient address
    function transferUnchecked(address recipient, uint256 amount) public {    infinite gas
        // No validation of recipient address (e.g., ensuring it's not the zero address)
        _transfer(msg.sender, recipient, amount);
    }
}
```

BinaryOperation

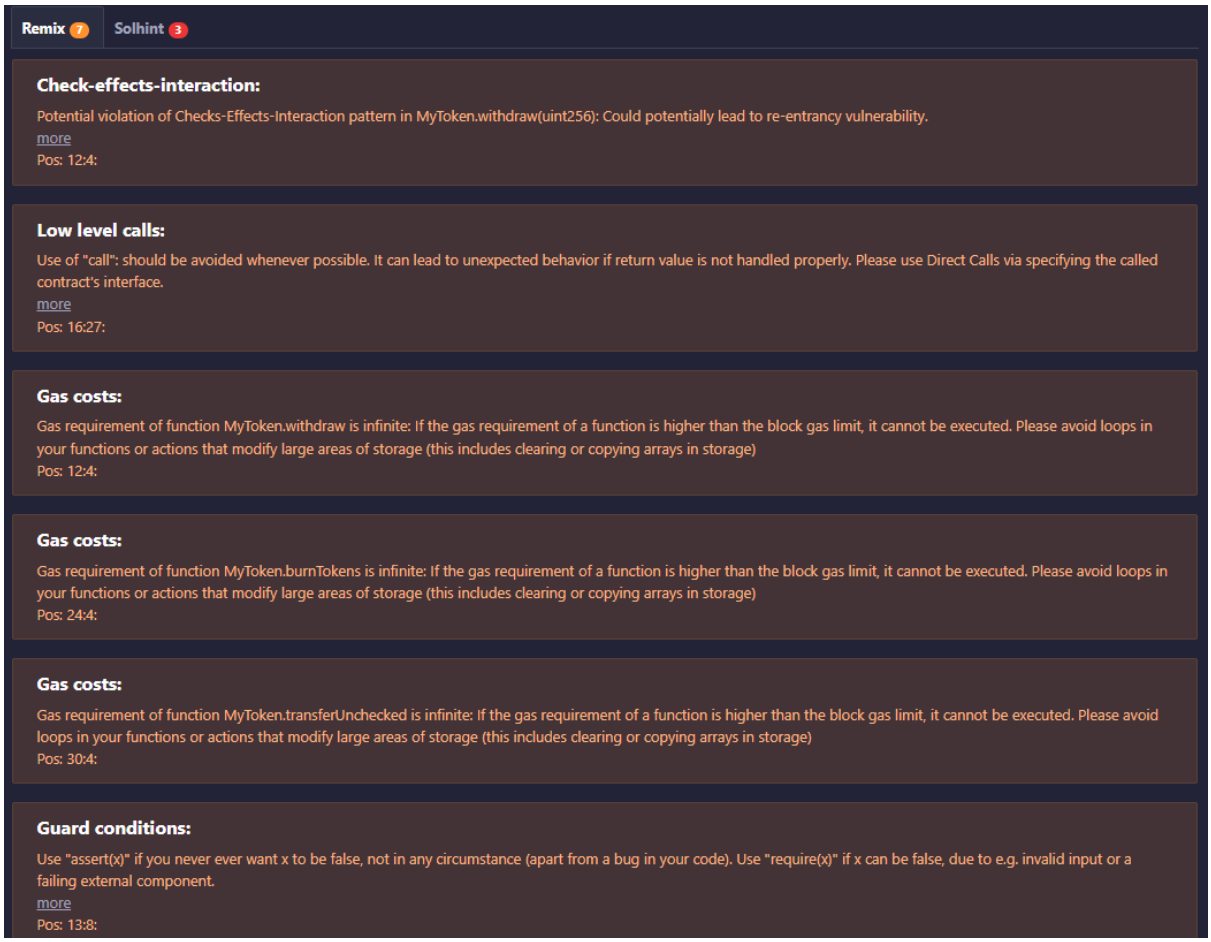
exp2.sol 12:16

2. Use Remix Solidity Analyzers Plugin

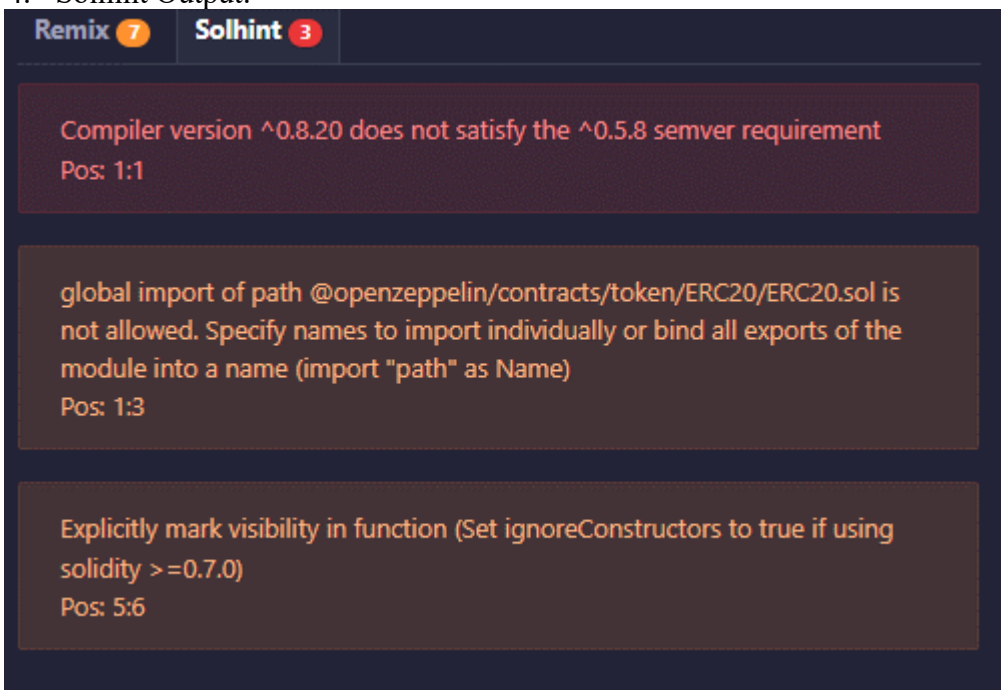


3. Compile the code and analyse the vulnerabilities

Name: Abhishek Pandey SAP ID: 60019220110
Roll No: B073 Branch: CSE(ICB)



4. Solhint Output:



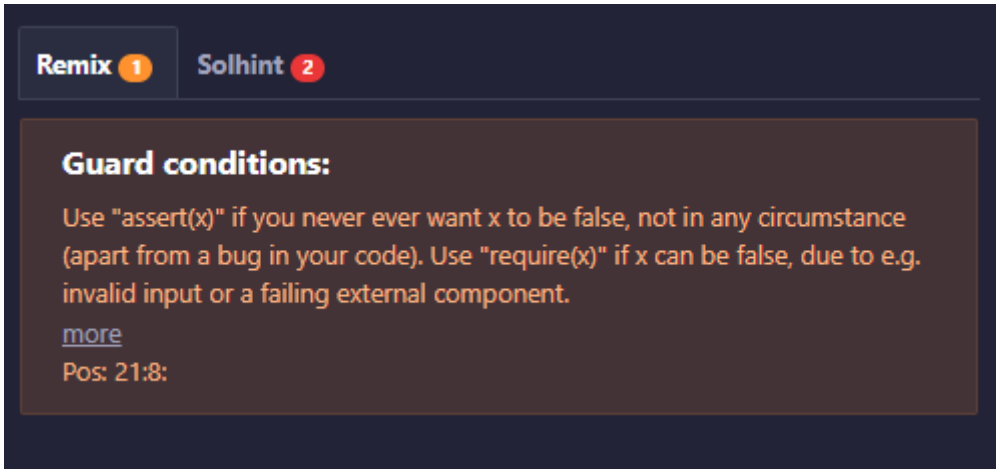
Name: Abhishek Pandey
SAP ID: 60019220110
Roll No: B073
Branch: CSE(ICB)

Q 2. Implement Access Control Vulnerability:

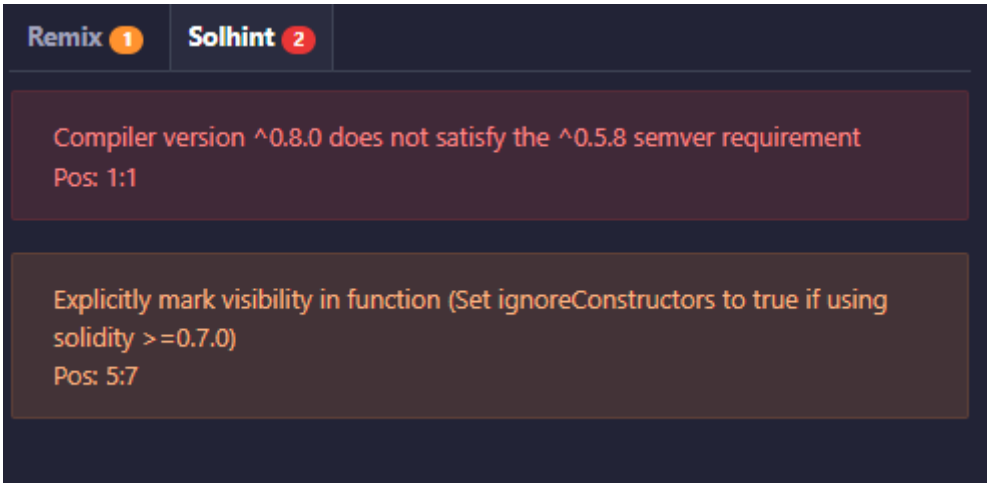
Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract VulnerableAccessControl {
    address public owner;
    uint256 public secretValue;

    constructor() {    206178 gas 159600 gas
        owner = msg.sender;
        secretValue = 42;
    }

    // This function should only be callable by the owner,
    // but there is NO access control here!
    function setSecretValue(uint256 _value) public {    22515 gas
        secretValue = _value;
    }

    // A safe function with proper access control for contrast
    function safeSetSecretValue(uint256 _value) public {    24661 gas
        require(msg.sender == owner, "Not the owner");
        secretValue = _value;
    }
}
```

Output:



Name: Abhishek Pandey
SAP ID: 60019220110
Roll No: B073
Branch: CSE(ICB)



Q 3. Implement Fixed access control contract:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract FixedAccessControl {
    address public owner;
    mapping(address => uint256) public balances;

    // Set the deployer as the owner on contract creation
    constructor() {    561830 gas 537000 gas
        owner = msg.sender;
    }

    // Modifier to restrict function access only to the owner
    modifier onlyOwner() {
        require(msg.sender == owner, "Access denied: Only owner allowed");
        _;
    }

    // Example owner-only function: Mint tokens to an address
    function mint(address to, uint256 amount) public onlyOwner {    infinite gas
        require(to != address(0), "Cannot mint to zero address");
        balances[to] += amount;
    }

    // Transfer tokens from sender to recipient
    function transfer(address recipient, uint256 amount) public returns (bool) {    infinite gas
        require(balances[msg.sender] >= amount, "Insufficient balance");
        require(recipient != address(0), "Cannot transfer to zero address");

        balances[msg.sender] -= amount;
        balances[recipient] += amount;
        return true;
    }
```

Name: Abhishek Pandey
Roll No: B073
SAP ID: 60019220110
Branch: CSE(ICB)

```
    // Check the balance of an address
    function balanceOf(address account) public view returns (uint256) {    2851 gas
        return balances[account];
    }

    // Transfer ownership to a new owner
    function transferOwnership(address newOwner) public onlyOwner {    27004 gas
        require(newOwner != address(0), "New owner cannot be zero address");
        owner = newOwner;
    }
}
```

expq3.sol 40:66

Output:

Remix 7 Solhint 3

Gas costs:

Gas requirement of function FixedAccessControl.mint is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)
Pos: 20:4:

Gas costs:

Gas requirement of function FixedAccessControl.transfer is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)
Pos: 26:4:

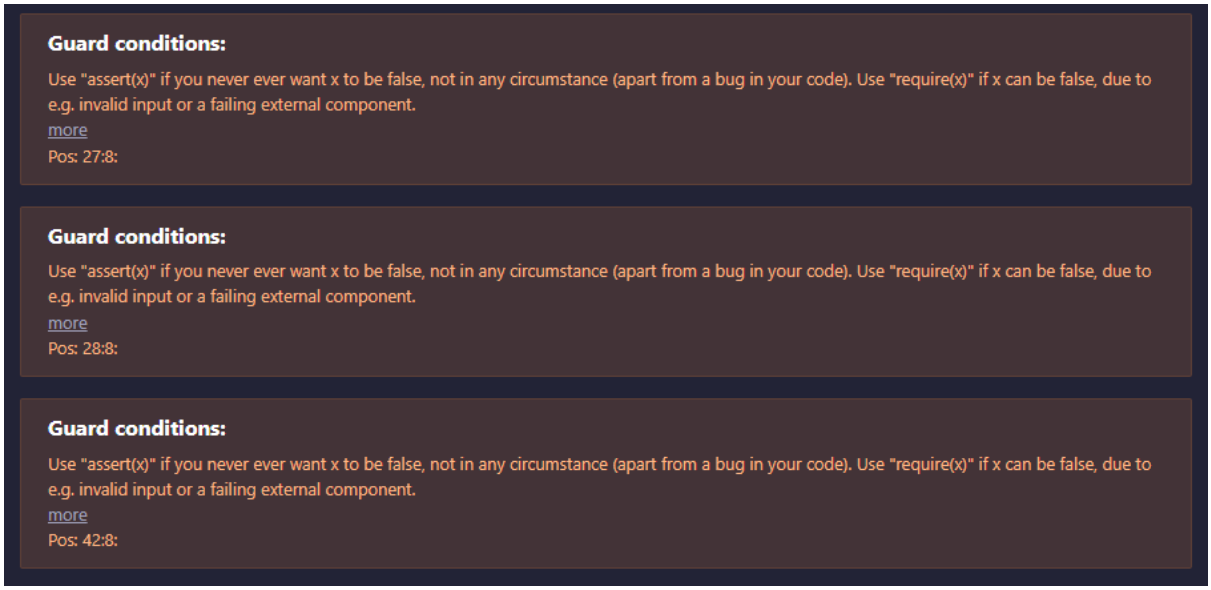
Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.
more
Pos: 15:8:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.
more
Pos: 21:8:

Name: Abhishek Pandey
Roll No: B073
SAP ID: 60019220110
Branch: CSE(ICB)

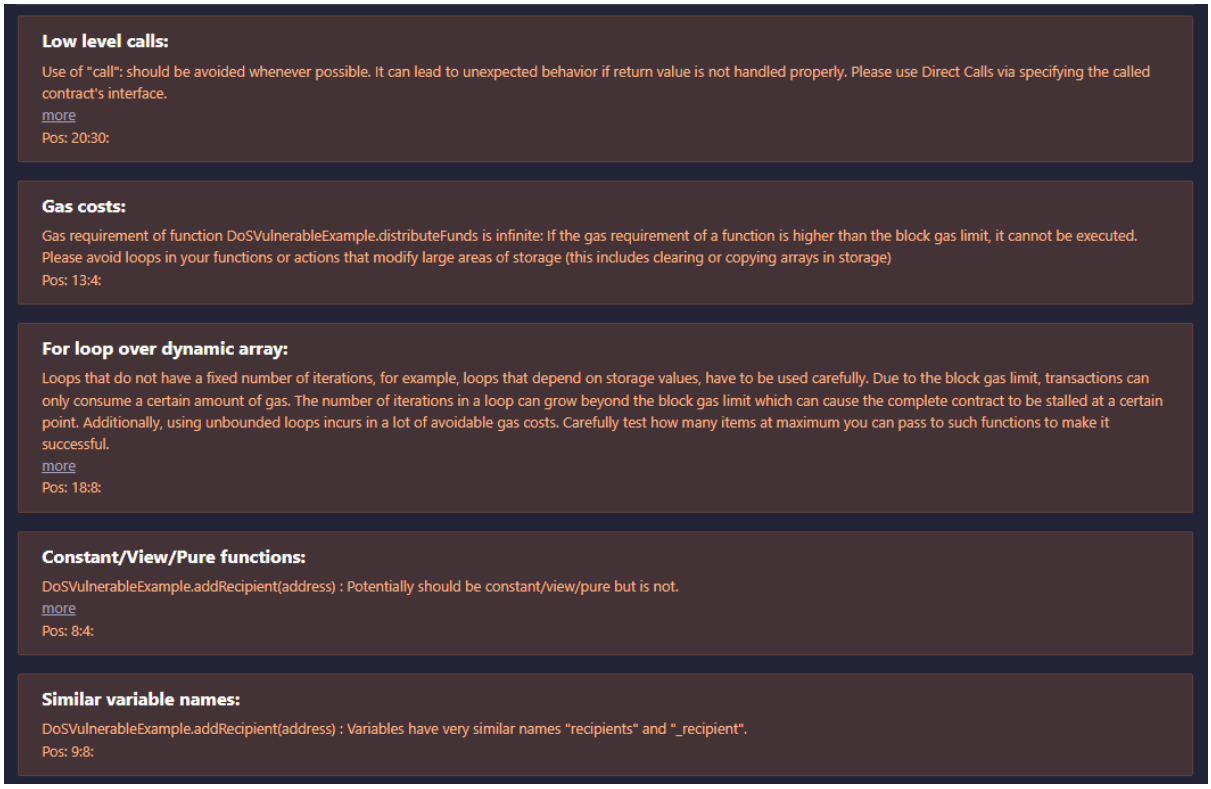


Q 4: Implement Denial of Service Vulnerability:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract DoSVulnerableExample {
    address[] public recipients;

    // Add a recipient address
    function addRecipient(address _recipient) public {    49002 gas
        recipients.push(_recipient);
    }

    // Distribute Ether equally to all recipients
    function distributeFunds() public payable {    infinite gas
        require(msg.value > 0, "Send some ether");

        uint256 share = msg.value / recipients.length;

        for (uint i = 0; i < recipients.length; i++) {
            // If one recipient's transfer fails, whole transaction reverts — causes DoS
            (bool sent, ) = recipients[i].call{value: share}("");
            require(sent, "Transfer to recipient failed");
        }
    }
}
```

Outputs:

Name: Abhishek Pandey
Roll No: B073
SAP ID: 60019220110
Branch: CSE(ICB)



Q 5: Implement Fixed auction contract for DoS:

Name: Abhishek Pandey SAP ID: 60019220110
Roll No: B073 Branch: CSE(ICB)

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract FixedAuction {
    address public owner;
    uint public auctionEndTime;

    address public highestBidder;
    uint public highestBid;

    // Keep track of pending returns for each bidder
    mapping(address => uint) public pendingReturns;

    bool public ended;

    event HighestBidIncreased(address bidder, uint amount);
    event AuctionEnded(address winner, uint amount);

    constructor(uint _biddingTime) {    infinite gas 509800 gas
        owner = msg.sender;
        auctionEndTime = block.timestamp + _biddingTime;
    }

    // Bid function where bidders send ETH
    function bid() public payable {    infinite gas
        require(block.timestamp <= auctionEndTime, "Auction already ended");
        require(msg.value > highestBid, "There already is a higher bid");

        if (highestBid != 0) {
            // Instead of sending refund directly, record it for withdrawal
            pendingReturns[highestBidder] += highestBid;
        }

        highestBidder = msg.sender;
        highestBid = msg.value;
```

Name: Abhishek Pandey
Roll No: B073
SAP ID: 60019220110
Branch: CSE(ICB)

```
        emit HighestBidIncreased(msg.sender, msg.value);
    }

    // Withdraw function for bidders to retrieve their refunds
    function withdraw() public returns (bool) {    infinite gas
        uint amount = pendingReturns[msg.sender];
        if (amount > 0) {
            // Reset before sending to prevent re-entrancy attacks
            pendingReturns[msg.sender] = 0;

            if (!payable(msg.sender).send(amount)) {
                // If send fails, restore the amount
                pendingReturns[msg.sender] = amount;
                return false;
            }
        }
        return true;
    }

    // End the auction and send funds to owner
    function auctionEnd() public {    infinite gas
        require(block.timestamp >= auctionEndTime, "Auction not yet ended");
        require(!ended, "auctionEnd already called");

        ended = true;
        emit AuctionEnded(highestBidder, highestBid);

        // Transfer highest bid to the owner
        payable(owner).transfer(highestBid);
    }
}
```

Output:

Name: Abhishek Pandey
SAP ID: 60019220110
Roll No: B073
Branch: CSE(ICB)

Check-effects-interaction:

Potential violation of Checks-Effects-Interaction pattern in FixedAuction.withdraw(): Could potentially lead to re-entrancy vulnerability.
more
Pos: 40:4:

Block timestamp:

Use of "block.timestamp": "block.timestamp" can be influenced by miners to a certain degree. That means that a miner can "choose" the block.timestamp, to a certain degree, to change the outcome of a transaction in the mined block.
more
Pos: 21:25:

Block timestamp:

Use of "block.timestamp": "block.timestamp" can be influenced by miners to a certain degree. That means that a miner can "choose" the block.timestamp, to a certain degree, to change the outcome of a transaction in the mined block.
more
Pos: 26:16:

Block timestamp:

Use of "block.timestamp": "block.timestamp" can be influenced by miners to a certain degree. That means that a miner can "choose" the block.timestamp, to a certain degree, to change the outcome of a transaction in the mined block.
more
Pos: 57:16:

Low level calls:

Use of "send": "send" does not throw an exception when not successful, make sure you deal with the failure case accordingly. Use "transfer" whenever failure of the ether transfer should rollback the

Name: Abhishek Pandey SAP ID: 60019220110
Roll No: B073 Branch: CSE(ICB)

Use of "send": "send" does not throw an exception when not successful, make sure you deal with the failure case accordingly. Use "transfer" whenever failure of the ether transfer should rollback the whole transaction. Note: if you "send/transfer" ether to a contract the fallback function is called, the callees fallback function is very limited due to the limited amount of gas provided by "send/transfer". No state changes are possible but the callee can log the event or revert the transfer. "send/transfer" is syntactic sugar for a "call" to the fallback function with 2300 gas and a specified ether value.
more
Pos: 46:17:

Gas costs:

Gas requirement of function FixedAuction.bid is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)
Pos: 25:4:

Gas costs:

Gas requirement of function FixedAuction.withdraw is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)
Pos: 40:4:

Gas costs:

Gas requirement of function FixedAuction.auctionEnd is infinite: If the gas requirement of a function is higher than the block gas limit, it cannot be executed. Please avoid loops in your functions or actions that modify large areas of storage (this includes clearing or copying arrays in storage)
Pos: 56:4:

Guard conditions:

Use "assert(x)" if you never ever want x to be false, not in any circumstance (apart from a bug in your code). Use "require(x)" if x can be false, due to e.g. invalid input or a failing external component.
more
Pos: 26:8:

Name: Abhishek Pandey
Roll No: B073
SAP ID: 60019220110
Branch: CSE(ICB)

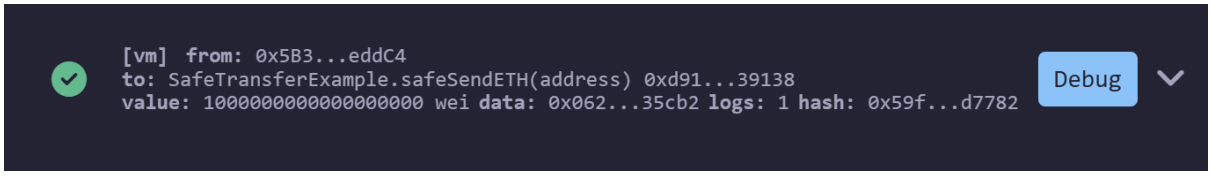
Q 6: Implement Unchecked Return Value Vulnerability

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/// @title SafeTransferExample
/// @notice Demonstrates secure handling of return values from external calls
contract SafeTransferExample {
    event EthSent(address indexed to, uint256 amount);
    event TokenSent(address indexed token, address indexed to, uint256 amount);

    /// @notice Securely send ETH to `to`. Reverts if the send failed.
    function safeSendETH(address payable to) external payable {    undefined gas
        require(msg.value > 0, "No ETH provided");

        // Use call and check its returned success boolean
        (bool success, ) = to.call{value: msg.value}("");
        require(success, "ETH transfer failed");

        emit EthSent(to, msg.value);
    }

    /// @notice Secure wrapper for ERC20 transferFrom-like calls.
    /// @dev Many ERC20s return bool on transfer; some don't. We handle both.
    function safeTransferERC20(address token, address to, uint256 amount) external
```

```solidity
    /// @dev Many ERC20s return bool on transfer; some don't. We handle both.
    function safeTransferERC20(address token, address to, uint256 amount) external
        // bytes4(keccak256("transfer(address,uint256)")) == 0xa9059cbb
        (bool success, bytes memory data) =
            token.call(abi.encodeWithSelector(0xa9059cbb, to, amount));

        // If `success` is false we revert
        require(success, "Token call reverted");

        // If data is returned, ensure it decodes to true (some tokens return bool)
        if (data.length > 0) {
            // decode and ensure it is true
            require(abi.decode(data, (bool)), "Token transfer returned false");
        }

        emit TokenSent(token, to, amount);
    }

    // Fallback to receive ETH if someone sends ETH directly
    receive() external payable {}    undefined gas
}
```

Name: Abhishek Pandey SAP ID: 60019220110
Roll No: B073 Branch: CSE(ICB)

Output:

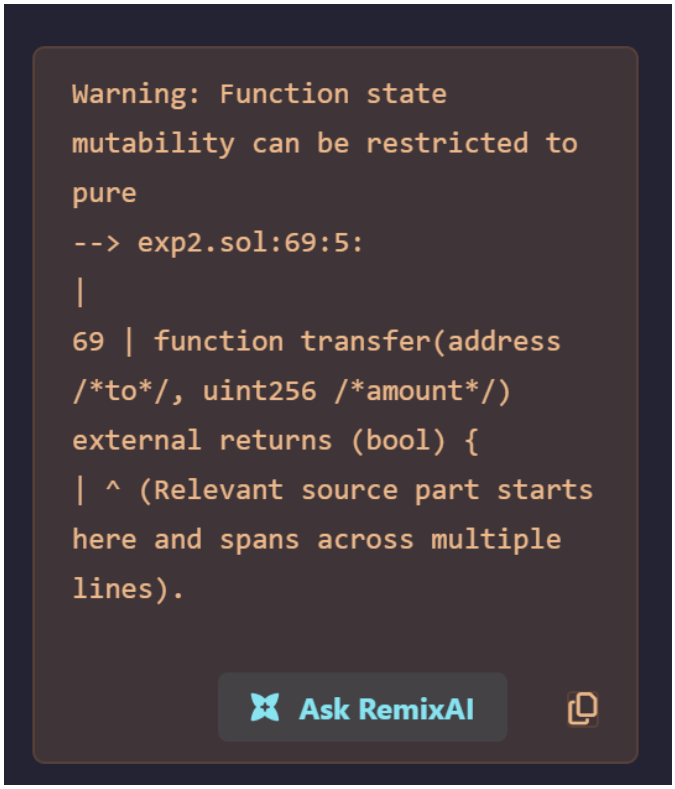


Q 7: Implement Fixed return value unchecked contract:

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/// @title UncheckedReturnExample
/// @notice Demonstrates a contract that ignores the return value of ERC20 transf
interface IERC20 {
    function transfer(address to, uint256 amount) external returns (bool);
}

contract UncheckedReturnExample {
    event TransferAttempt(address token, address to, uint256 amount);

    /// @notice Calls transfer but does NOT check the returned bool
    function sendTokens(address token, address to, uint256 amount) external {
        // ⚠️ Vulnerable: ignoring the return value
        IERC20(token).transfer(to, amount);

        emit TransferAttempt(token, to, amount);
    }
}

```

Output:

Name: Abhishek Pandey
Roll No: B073
SAP ID: 60019220110
Branch: CSE(ICB)



Conclusion:

Thus we successfully conducted code reviews and static analysis to identify potential security flaws such as re-entrancy bugs, integer overflows, or unchecked user inputs in smart contract.